Università degli Studi di Salerno

Dipartimento di Informatica

Corso di Laurea in Informatica

Progetto di Machine Learning

Anno Accademico 2024–2025

Signify

Voices Through Hands

Sistema di Sign Language Recognition Real-Time
basato su Computer Vision e Deep Learning

Docenti

Prof. Giuseppe Polese

Prof.ssa Loredana Caruccio

Studente

Antonio Walter De Fusco

Matricola: 0512119006

Febbraio 2025

Indice

1 Introduzione **1**

1.1 Il Problema della Comunicazione tra Sordi e Udenti . . . 1

1.2 Situazione Attuale nel Sign Language Recognition . . . 1

1.2.1 Approcci Tradizionali . . . 2

1.2.2 Approcci Vision-Based con Deep Learning . . . 2

1.2.3 Strumenti e Framework Chiave . . . 3

1.3 Perché Signify . . . 3

1.3.1 La Visione del Progetto . . . 3

1.3.2 Accessibilità e Scelte Tecnologiche . . . 4

1.3.3 Un Progetto Open-Source . . . 4

2 Il Dataset: ASL Citizen **5**

2.1 Creazione del Dataset . . . 5

2.2 Composizione . . . 5

2.3 Attendibilità . . . 5

3 Preprocessing: dai Video ai Punti Chiave **7**

3.1 L'Idea di Base . . . 7

3.2 MediaPipe Holistic . . . 7

3.3 Il Processo di Estrazione . . . 8

3.4 Parallelizzazione . . . 8

3.5 Risultati dell'Estrazione . . . 9

4 Analisi e Bilanciamento del Dataset **10**

4.1 Composizione Attuale del Dataset . . . 10

4.1.1 Distribuzione dei Campioni per Classe . . . 10

4.1.2 Landmark Mancanti: Prima Osservazione . . . 12

4.2 Analisi di Coerenza per Classe e per Signer . . . 13

4.2.1 Fase 1: Pattern Dominante per Classe . . . 13

4.2.2 Fase 2: Il Problema della Mano Dominante . . . 14

4.2.3 Classificazione degli Errori . . . 14

4.3 Pulizia del Dataset . . . 15

4.4 Bilanciamento del Dataset . . . 16

5 Scelta dell'Architettura del Modello **18**

5.1 Perché gli Approcci Tradizionali Non Sono Adatti . . . 18

5.2 Reti Neurali per Sequenze Temporali . . . 18

5.2.1 LSTM – Long Short-Term Memory . . . 18

5.2.2 Transformer . . . 19

5.2.3 LSTM con Meccanismo di Attention 20
5.3 Confronto e Scelta Finale 21

6 Implementazione del Modello **22**
6.1 Preparazione dei Dati (Dataset Pipeline) 22
6.2 L'Architettura LSTMAttention 22
6.2.1 Lo Strato di Attention (AttentionLayer) 22
6.2.2 Il Modello Principale (LSTMAttention) 24
6.3 Processo di Addestramento (Training Loop) 25

7 Risultati e Valutazione del Modello **27**
7.1 Metriche di Classificazione e Analisi dei Risultati 27
7.1.1 Analisi delle prestazioni 28
7.2 Analisi della Matrice di Confusione Multi-Classe (One-vs-Rest) 28
7.3 Ottimizzazione dei Risultati: Integrazione NLP 30
7.3.1 Ottimizzazione tramite Language Model (BERT Rescoring) 30

8 Messa in Produzione e Interfaccia Utente **32**
8.1 Architettura del Frontend (Vite + React) 32
8.2 Responsive Design e Prova su Dispositivi 32
8.3 Strategia di Deploy 34

9 Conclusioni e Sviluppi Futuri **35**
9.1 Sviluppi Futuri 35

Ringraziamenti **35**

1 Introduzione

1.1 Il Problema della Comunicazione tra Sordi e Udenti

Secondo l'Organizzazione Mondiale della Sanità (OMS) [1], oltre il 5% della popolazione mondiale — circa 430 milioni di persone — convive con una perdita uditiva disabilitante, di cui 34 milioni sono bambini. Le proiezioni indicano che entro il 2050 quasi 2,5 miliardi di individui presenteranno un qualche grado di perdita uditiva, e oltre 700 milioni necessiteranno di riabilitazione [1]. Il costo economico globale della perdita uditiva non trattata è stimato in circa 1 trilione di dollari all'anno, considerando spese sanitarie, supporto educativo, perdite di produttività e impatti sociali [2].

La **Lingua dei Segni** rappresenta il mezzo di comunicazione primario per la comunità sorda. Esistono oltre 300 lingue dei segni nel mondo, ciascuna con una propria grammatica, sintassi e struttura linguistica autonoma, distinta dalle corrispondenti lingue parlate [3]. La Lingua dei Segni Americana (ASL), utilizzata da circa mezzo milione di persone negli Stati Uniti e in Canada, è una delle più studiate e documentate.

Nonostante la sua ricchezza espressiva, la lingua dei segni crea una significativa **barriera comunicativa** tra la comunità sorda e la maggioranza della popolazione udente:

- **Ambito sanitario:** i pazienti sordi affrontano frequentemente incomprensioni, ritardi diagnostici e difficoltà nell'accesso alle cure, a causa della carenza di interpreti professionisti e di personale sanitario competente nella lingua dei segni [4].

- **Ambito lavorativo:** il 62% delle persone sorde e il 66% delle persone con problemi d'udito riportano limitazioni nelle opportunità professionali dovute a barriere comunicative [5].

- **Ambito educativo:** nei paesi in via di sviluppo, i bambini con perdita uditiva raramente ricevono un'istruzione adeguata, compromettendo le loro prospettive future [1].

- **Vita quotidiana:** interazioni apparentemente semplici — una telefonata, una conversazione informale, un'emergenza — diventano ostacoli significativi in assenza di strumenti di mediazione.

In questo contesto, la tecnologia può svolgere un ruolo fondamentale nell'abbattere queste barriere, rendendo la comunicazione più accessibile e inclusiva.

1.2 Situazione Attuale nel Sign Language Recognition

Il *Sign Language Recognition* (SLR) è un campo di ricerca interdisciplinare che combina Computer Vision, Natural Language Processing e Deep Learning per tradurre automaticamente i gesti della lingua dei segni in testo o parlato.

1.2.1 Approcci Tradizionali

I primi sistemi di riconoscimento della lingua dei segni si basavano su **dispositivi indossabili**: guanti sensorizzati dotati di accelerometri, giroscopi e sensori di flessione, capaci di catturare con precisione la posizione e il movimento delle dita [6]. Sebbene questi sistemi raggiungano ottime prestazioni teoriche, presentano limiti significativi:

- **Invasività:** l'utente deve indossare hardware dedicato, riducendo la naturalezza della comunicazione.
- **Costo:** i dispositivi professionali hanno costi proibitivi per un'adozione su larga scala.
- **Scalabilità:** la calibrazione è specifica per ogni utente e ogni dispositivo.

1.2.2 Approcci Vision-Based con Deep Learning

L'evoluzione delle reti neurali profonde e la crescente potenza computazionale hanno permesso lo sviluppo di sistemi **vision-based**, che utilizzano semplici webcam o fotocamere per catturare i gesti, eliminando la necessità di hardware dedicato [7].

Le architetture principali impiegate nel campo sono:

- **Convolutional Neural Networks (CNN):** utilizzate per l'estrazione di feature spaziali da singoli frame, efficaci nel riconoscimento di gesti statici come il *fingerspelling* (alfabeto manuale) [8]. Varianti come MobileNetV2 [9] consentono l'inferenza su dispositivi a risorse limitate.
- **Recurrent Neural Networks (RNN/LSTM):** le reti *Long Short-Term Memory* [10] eccellono nell'elaborazione di sequenze temporali, catturando la dinamica del movimento che caratterizza i segni. Le architetture ibride **CNN-LSTM** combinano l'estrazione spaziale con l'analisi temporale per dataset di segni isolati [6].
- **Transformer e Attention Mechanisms:** modelli più recenti basati su meccanismi di attenzione [11] stanno emergendo come alternativa promettente, in particolare per il riconoscimento continuo (*Continuous Sign Language Recognition*, CSLR), superando le combinazioni CNN-RNN nel riconoscimento di sequenze gestuali complesse [7].
- **Object Detection (YOLO):** algoritmi come YOLOv8 [12] vengono impiegati per la classificazione in tempo reale di lettere e gesti, offrendo un buon compromesso tra velocità e prestazioni [13].

1.2.3 Strumenti e Framework Chiave

L'ecosistema tecnologico attuale offre diversi strumenti fondamentali per lo sviluppo di sistemi SLR:

- **MediaPipe Holistic (Google):** framework open-source per il tracciamento simultaneo di pose del corpo, mani e volto in tempo reale [14]. Estrae fino a 543 landmark 3D per frame, fornendo una rappresentazione compatta e ricca del gesto. È diventato lo standard de facto per l'estrazione di feature nella ricerca SLR.
- **OpenCV:** libreria di Computer Vision [15] utilizzato per l'acquisizione e il preprocessing del segnale video, la manipolazione dei frame e l'integrazione con modelli di inferenza.
- **PyTorch:** framework di Deep Learning [16] per la costruzione, l'addestramento e il deployment di modelli neurali. La sua natura dinamica e l'ampia adozione nella ricerca lo rendono lo strumento ideale per architetture LSTM complesse.
- **ONNX Runtime Web:** libreria che consente l'esecuzione di modelli di Machine Learning direttamente nel browser con accelerazione hardware [17], abilitando applicazioni web senza necessità di server dedicati.

1.3 Perché Signify

Date le barriere comunicative descritte e il panorama tecnologico attuale, nasce l'esigenza di uno strumento che sia al contempo tecnicamente efficace e concretamente accessibile. **Signify — Voices Through Hands** è la risposta a questa esigenza.

1.3.1 La Visione del Progetto

Signify nasce con un obiettivo chiaro: **dare voce ai gesti**. Il sistema traduce il segnale video catturato da una semplice webcam in testo attraverso una pipeline che combina Computer Vision e Deep Learning:

$$\textit{Video Webcam} \longrightarrow \textbf{MediaPipe} \text{ (Estrazione Landmark)} \longrightarrow \textbf{LSTM} \text{ (Classificazione Temporale)} \longrightarrow \textit{Output Testuale}$$

Il progetto si concentra sulla Lingua dei Segni Americana (ASL) come punto di partenza, utilizzando il dataset *ASL Citizen* [18], con l'ambizione di estendere il supporto ad altre lingue dei segni in futuro.

Dal punto di vista del Machine Learning, il problema viene modellato come un task di **Classificazione Multiclasse su Serie Temporali**:

- **Input:** una sequenza di frame video $X = \{x_1, x_2, \ldots, x_T\}$ contenenti un gesto.

- **Output:** una classe $y \in Y$ corrispondente alla parola (*gloss*) eseguita.
- **Feature:** vettori di 258 dimensioni per frame (132 landmark della posa, 63 per ciascuna mano), estratti tramite MediaPipe Holistic [14].

1.3.2 Accessibilità e Scelte Tecnologiche

A differenza delle soluzioni commerciali chiuse — come DeepASL [19], SignLink o SignSpeak — che presentano licenze costose e API proprietarie, Signify adotta un approccio radicalmente diverso.

Il sistema è progettato con una filosofia *"low-barrier"*: richiede unicamente una webcam standard e un computer con risorse computazionali modeste. Non sono necessari sensori di profondità, guanti sensorizzati o GPU dedicate per l'inferenza. Questa scelta progettuale si traduce in:

- **Hardware minimo:** una webcam integrata è sufficiente per l'acquisizione video.
- **Deployment flessibile:** l'architettura è progettata per il funzionamento sia locale che via browser.
- **Stack tecnologico aperto:** MediaPipe [14], PyTorch e Python sono tutti strumenti gratuiti e ben documentati.

1.3.3 Un Progetto Open-Source

Signify è rilasciato come progetto **completamente open-source**, con il codice sorgente disponibile pubblicamente su GitHub. Questa scelta non è casuale, ma risponde a un principio fondamentale: *uno strumento nato per abbattere barriere non deve crearne di nuove*. L'approccio open-source garantisce che:

- Chiunque possa utilizzare, modificare e distribuire il software gratuitamente.
- Le istituzioni educative possano integrarlo nei percorsi formativi senza costi di licenza.
- Le organizzazioni della comunità sorda possano adattarlo alle proprie esigenze specifiche.
- La comunità di sviluppatori possa contribuire al miglioramento continuo del sistema.

Nei capitoli successivi verranno descritti in dettaglio il dataset utilizzato, la pipeline di estrazione e preprocessamento dei dati, e l'architettura del modello di classificazione.

2 Il Dataset: ASL Citizen

Per l'addestramento del modello di Signify è stato selezionato il dataset **ASL Citizen** [18], il più grande dataset pubblico per il riconoscimento di segni isolati (*Isolated Sign Language Recognition*, ISLR) nella Lingua dei Segni Americana.

2.1 Creazione del Dataset

ASL Citizen è stato sviluppato da un team di ricercatori della **Microsoft Research** e di diverse università americane, con il coinvolgimento attivo di membri sordi nel gruppo di lavoro [18].

La raccolta dei dati è avvenuta tramite una **piattaforma di crowdsourcing** appositamente progettata: i firmatari, tutti sordi o con problemi d'udito, hanno registrato i propri video autonomamente nei loro ambienti domestici. Per ogni segno, i partecipanti visualizzavano prima un video dimostrativo eseguito da un modello ASL altamente competente, e successivamente registravano la propria versione del segno.

Il vocabolario dei segni è stato standardizzato utilizzando **ASL-LEX** [20], un database linguistico che fornisce identificatori univoci e analisi fonologica per ogni segno, eliminando ambiguità nelle annotazioni.

2.2 Composizione

Il dataset contiene:

- **83.399 video** di segni isolati.
- **2.731 segni distinti** (*gloss*).
- **52 firmatari** sordi o con problemi d'udito, provenienti da 16 stati americani, con un'esperienza nell'uso dell'ASL compresa tra 2 e 65 anni [18].

Essendo stato raccolto in ambienti reali e non in laboratorio, il dataset presenta una naturale varietà di condizioni di illuminazione, sfondi, risoluzioni e angolazioni della webcam — caratteristiche essenziali per addestrare un modello robusto in contesti d'uso reali.

Il dataset include split predefiniti per training, validation e test che sono **signer-independent**: i firmatari nel set di test non compaiono mai nel training [18]. Questo garantisce che il modello venga valutato sulla capacità di riconoscere segni eseguiti da persone mai viste durante l'addestramento. Gli split sono inoltre stratificati per genere.

2.3 Attendibilità

L'attendibilità del dataset è supportata da diversi fattori:

1. **Pubblicazione peer-reviewed:** il dataset è stato presentato alla conferenza **NeurIPS** (*Advances in Neural Information Processing Systems*) [18], una delle sedi scientifiche più prestigiose nel campo del Machine Learning.

2. **Raccolta etica:** tutti i dati sono stati raccolti con **consenso informato** e il protocollo ha ricevuto l'approvazione di un **Institutional Review Board** (IRB) [18].

3. **Standardizzazione linguistica:** l'uso di ASL-LEX [20] come riferimento garantisce coerenza e precisione nelle annotazioni, a differenza di altri dataset che soffrono di etichette inconsistenti.

3 Preprocessing: dai Video ai Punti Chiave

Il dataset ASL Citizen è composto da video in formato MP4. Per poter addestrare un modello di Machine Learning, è necessario trasformare questi video in una rappresentazione numerica strutturata. In questa sezione viene descritto il processo di **estrazione dei landmark** (punti chiave), che converte ogni video in una sequenza di vettori numerici pronti per l'addestramento.

3.1 L'Idea di Base

Anziché lavorare direttamente sui pixel dei video — un approccio computazionalmente costoso e sensibile a variazioni di sfondo, illuminazione e risoluzione — si è scelto di rappresentare ogni frame come un insieme di **punti chiave** (*landmark*) del corpo umano. Ogni punto chiave ha coordinate (x, y, z) normalizzate, che descrivono la posizione di un'articolazione o un punto anatomico rilevante.

Questo approccio presenta due vantaggi fondamentali:

- **Riduzione della dimensionalità:** un frame video di 640×480 pixel contiene oltre 900.000 valori (RGB), mentre la rappresentazione a landmark produce soltanto 258 valori per frame.
- **Invarianza ambientale:** i landmark catturano esclusivamente la geometria del corpo, risultando indipendenti da sfondo, illuminazione, colore della pelle e vestiti.

3.2 MediaPipe Holistic

Per l'estrazione dei landmark è stata utilizzata la libreria **MediaPipe** [14], sviluppata da Google. In particolare, il modulo **Holistic** rileva simultaneamente tre componenti del corpo:

- **Pose (corpo):** 33 landmark, ognuno con 4 valori $(x, y, z, \textit{visibility})$ = **132 valori**
- **Mano sinistra:** 21 landmark, ognuno con 3 valori (x, y, z) = **63 valori**
- **Mano destra:** 21 landmark, ognuno con 3 valori (x, y, z) = **63 valori**

In totale, per ogni frame si ottiene un vettore di $132 + 63 + 63 =$ **258 valori**. Se un componente non viene rilevato in un frame (ad esempio, una mano non visibile), il vettore corrispondente viene riempito con zeri.

3.3 Il Processo di Estrazione

Il Jupyter notebook di estrazione (`Extract_Landmarks.ipynb`) elabora i video secondo i seguenti passi:

1. **Caricamento del CSV:** viene letto il file `train.csv` dal dataset ASL Citizen, che contiene per ogni riga il nome del file video e il *gloss* (etichetta) corrispondente.

2. **Estrazione dal ZIP:** i video sono archiviati in un file ZIP di grandi dimensioni. Per ogni video, il worker estrae temporaneamente il file MP4.

3. **Lettura frame per frame:** utilizzando OpenCV (`cv2.VideoCapture`), ogni frame del video viene letto e convertito da BGR a RGB (il formato richiesto da MediaPipe).

4. **Rilevamento dei landmark:** ogni frame viene processato da MediaPipe Holistic, che restituisce le coordinate dei 75 punti chiave (33 pose + 21 mano sinistra + 21 mano destra).

5. **Costruzione della sequenza:** i 258 valori estratti da ogni frame vengono concatenati in un array. Al termine del video, l'intera sequenza viene salvata come file `.npy` (formato binario NumPy).

6. **Organizzazione dell'output:** i file vengono salvati in cartelle nominate con il *gloss* corrispondente. Ad esempio, il video del segno "APPLE" viene salvato in `Dataset_Keypoints_Train/APPLE/`.

La funzione principale di estrazione è la seguente:

```python
def extract_keypoints(results):
    pose = np.array([[res.x, res.y, res.z, res.visibility]
        for res in results.pose_landmarks.landmark]).flatten() \
        if results.pose_landmarks else np.zeros(132)
    lh = np.array([[res.x, res.y, res.z]
        for res in results.left_hand_landmarks.landmark]).flatten() \
        if results.left_hand_landmarks else np.zeros(63)
    rh = np.array([[res.x, res.y, res.z]
        for res in results.right_hand_landmarks.landmark]).flatten() \
        if results.right_hand_landmarks else np.zeros(63)
    return np.concatenate([pose, lh, rh])
```

Listing 1: Funzione di estrazione dei landmark da un singolo frame

3.4 Parallelizzazione

Data la mole di video da elaborare, il notebook utilizza il modulo `multiprocessing` di Python per distribuire il lavoro su più core della CPU. Ogni *worker* elabora un video in

modo indipendente, inizializzando la propria istanza di MediaPipe Holistic. Il notebook supporta anche il **resume**: se un file `.npy` esiste già, il video corrispondente viene saltato, permettendo di riprendere l'elaborazione dopo un'interruzione.

3.5 Risultati dell'Estrazione

L'esecuzione completa ha prodotto i seguenti risultati:

Metrica	**Valore**
Video elaborati con successo	40.154
Errori di elaborazione	17
Video senza frame validi	31
Segni distinti (cartelle)	2.730
Dimensione vettore per frame	258 valori
Tempo totale di elaborazione	∼7 ore 24 minuti

La struttura finale dei dati è organizzata come segue:

```
Dataset_Keypoints_Train/
    APPLE/
        15890366051589533-APPLE.npy    (71 frame x 258 valori)
        654679171689897-APPLE.npy      (111 frame x 258 valori)
        ...
    HELLO/
        ...
    THANK_YOU/
        ...
    (2.730 cartelle totali)
```

Listing 2: Struttura delle cartelle di output

Ogni file `.npy` contiene un array NumPy di forma $(n_frame, 258)$, dove il numero di frame varia da video a video in base alla durata del segno. Questa variabilità nella lunghezza delle sequenze sarà gestita nella fase successiva di addestramento del modello.

4 Analisi e Bilanciamento del Dataset

Prima di procedere con l'addestramento di un modello di Machine Learning, è fondamentale analizzare la composizione del dataset estratto per identificare potenziali fonti di *bias*, sbilanciamenti e anomalie. Un dataset non curato può portare a fenomeni di *overfitting* (il modello memorizza i dati anziché generalizzare) o *underfitting* (il modello non riesce ad apprendere pattern significativi). In questa sezione viene presentata l'analisi completa dei dati estratti nel capitolo precedente e le operazioni di pulizia effettuate per garantire la qualità del dataset.

4.1 Composizione Attuale del Dataset

Il dataset di addestramento corrisponde al dataset estratto composto da **40.105 file .npy** distribuiti su **2.730 classi** (segni distinti), per un totale di **3.458.743 frame** analizzati. Attraverso un Jupyter notebook di analisi (`Analyze_Dataset.ipynb`) sono state ricavate le seguenti osservazioni.

4.1.1 Distribuzione dei Campioni per Classe

La distribuzione del numero di campioni per classe è riportata in Figura 1. Il dataset risulta relativamente bilanciato: la media è di **14.7 campioni per classe** con una deviazione standard di soli 1.4, e la mediana si attesta a 15. La maggior parte delle classi possiede tra 13 e 16 campioni.

Tuttavia, sono state individuate delle eccezioni rilevanti:

- **2 classi completamente vuote** (`HURDLE` e `THIS`), presumibilmente a causa di errori durante l'estrazione.
- **19 classi con 10 o meno campioni**, significativamente sotto la media.
- **5 classi con oltre 20 campioni**, sopra la media.

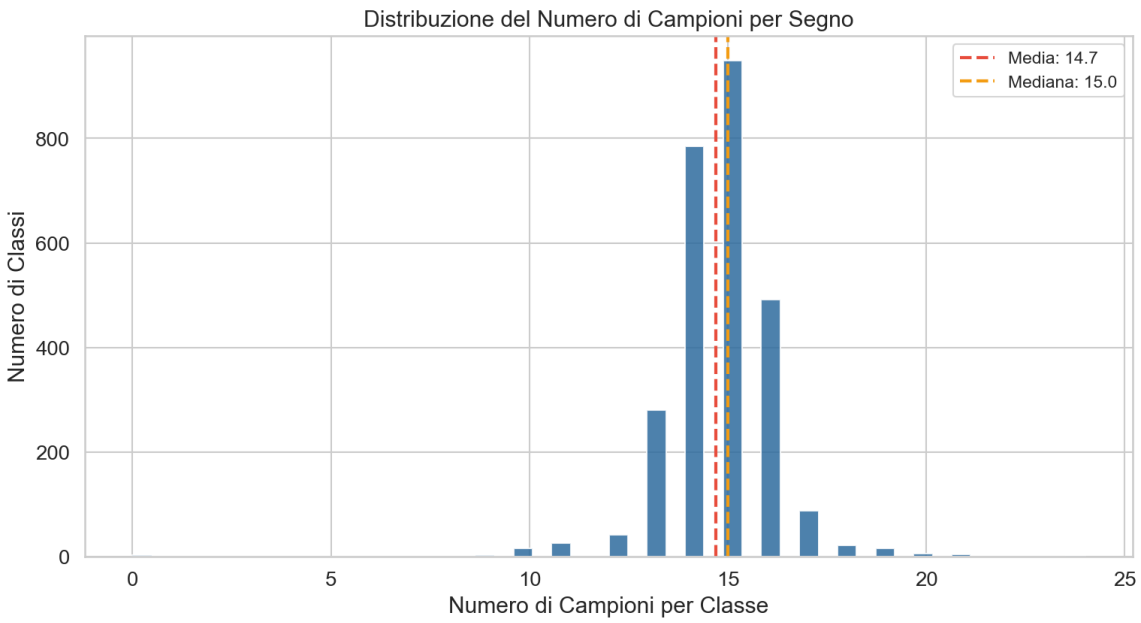

Figura 1: Distribuzione del numero di campioni per classe. La linea rossa indica la media (14.7), quella arancione la mediana (15).

Le classi con il minor e il maggior numero di campioni sono visualizzate in Figura 2. Le classi sotto-rappresentate possono introdurre *bias* nel modello, poiché il classificatore avrebbe meno esempi da cui apprendere i pattern di quei segni.

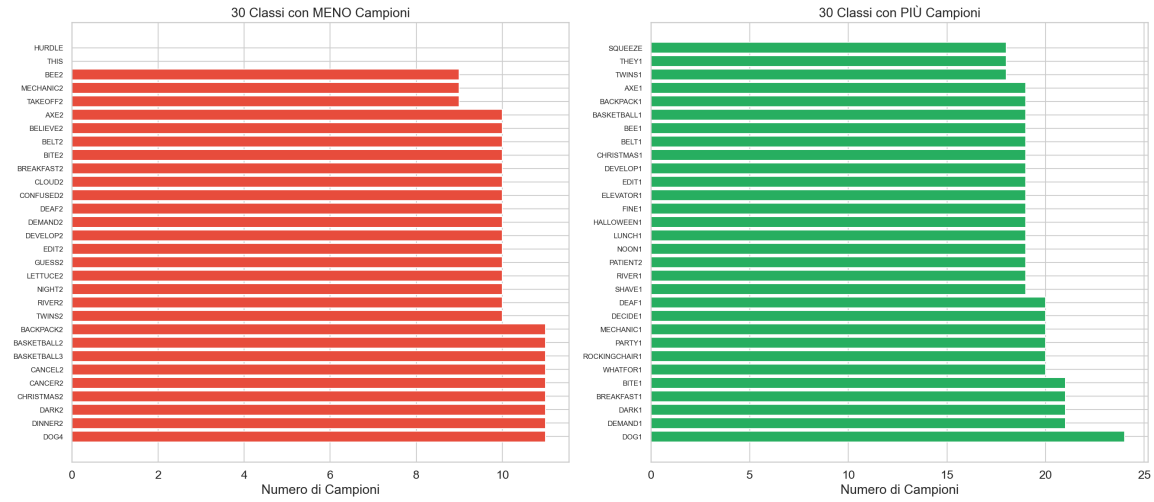

Figura 2: Le 30 classi con meno campioni (rosso, a sinistra) e le 30 classi con più campioni (verde, a destra).

L'analisi della durata temporale delle sequenze landmark (Figura 3) rivela una distribuzione asimmetrica con una marcata "coda lunga" verso destra. Sebbene la maggior parte dei segni abbia una durata contenuta (mediana di 75 frame), il dataset include esecuzioni eccezionalmente lunghe che raggiungono i 680 frame.

- **Valori centrali:** la media si attesta a 86.2 frame, con una concentrazione principale tra i 40 e i 100 frame ($\approx$ 1.5–3 secondi).
- **Orizzonte pratico (95° percentile):** il 95% dei campioni termina entro i 168 frame. Questo valore è cruciale per definire la strategia di campionamento: indica che una lunghezza target di 30 frame permette di catturare l'essenza dinamica del segno senza perdite eccessive di informazione, normalizzando la varianza tra segni rapidi e segni molto prolungati.

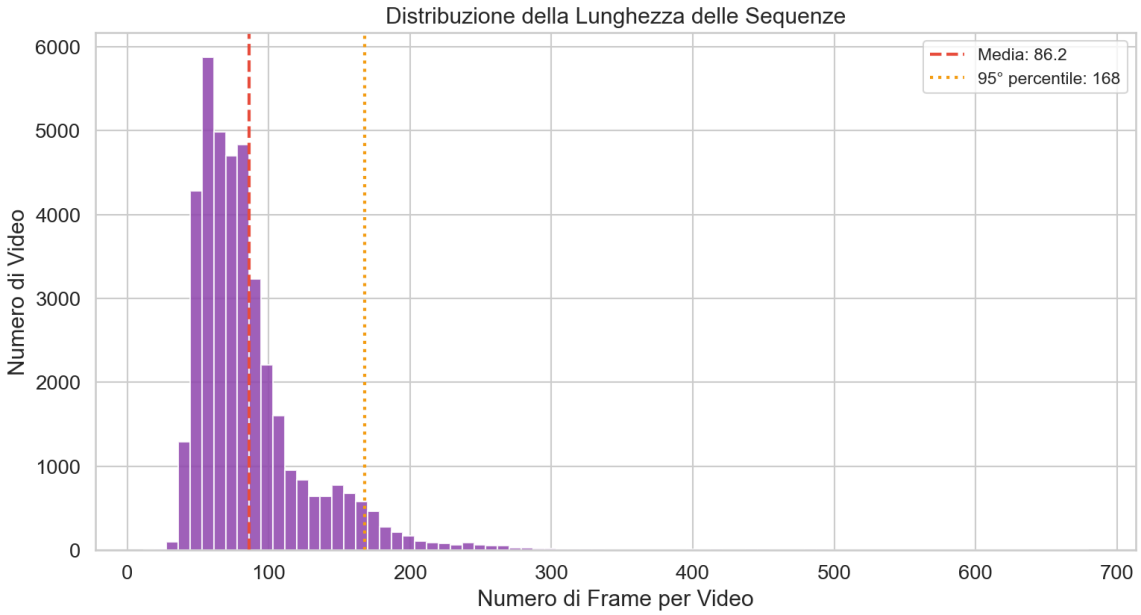

Figura 3: Analisi statistica della lunghezza dei video. L'istogramma evidenzia come la quasi totalità dei dati (95%) rientri sotto i 168 frame, supportando la scelta di una normalizzazione temporale fissa.

4.1.2 Landmark Mancanti: Prima Osservazione

L'analisi dei landmark mancanti (Figura 4) ha prodotto un risultato inizialmente allarmante:

- **Pose (corpo):** solo lo 0.1% dei frame presenta landmark a zero — un valore trascurabile.
- **Mano sinistra:** il 71.4% dei frame non presenta landmark rilevati.
- **Mano destra:** il 63.0% dei frame non presenta landmark rilevati.

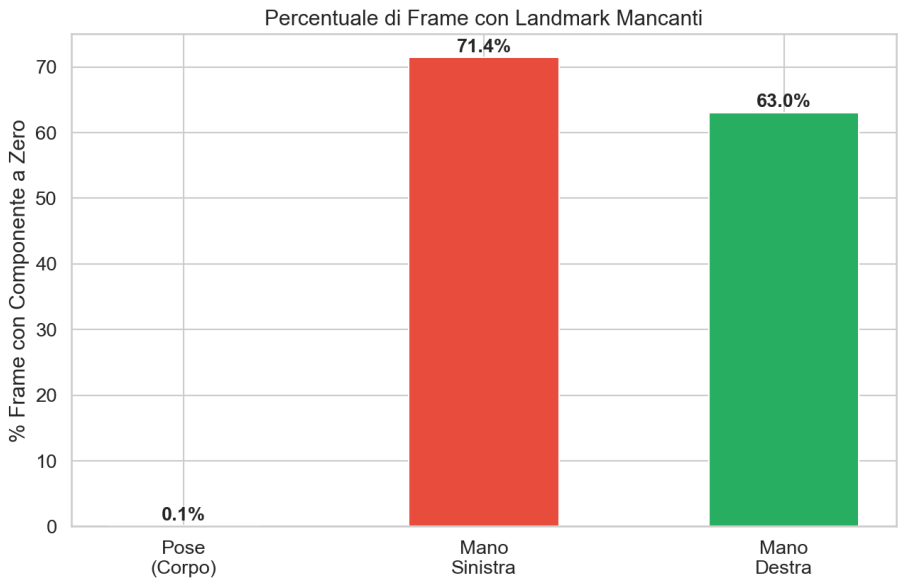

Figura 4: Percentuale di frame con componente landmark completamente a zero. Le mani risultano non rilevate nella maggioranza dei frame.

A prima vista, queste percentuali suggerirebbero un problema grave nel rilevamento. Tuttavia, riflettendo sulla natura della lingua dei segni americana, è stata formulata un'ipotesi diversa: **molti segni ASL sono eseguiti con una sola mano**. La mano non impegnata nel segno rimane naturalmente a riposo, e MediaPipe correttamente non rileva landmark per essa. La differenza tra mano sinistra (71.4%) e mano destra (63.0%) è coerente con questa ipotesi: la mano destra, essendo quella dominante per la maggior parte dei firmatari, viene attivata più frequentemente.

Questa osservazione ha portato allo sviluppo di un'analisi di coerenza più raffinata, descritta nella sezione successiva.

4.2 Analisi di Coerenza per Classe e per Signer

Per distinguere i **dati mancanti reali** (errori di MediaPipe) dai **dati mancanti attesi** (segni a una mano), è stata sviluppata un'analisi in due fasi: prima a livello di classe, poi a livello di signer.

4.2.1 Fase 1: Pattern Dominante per Classe

Per ogni classe (segno), ogni istanza è stata classificata in base al pattern di utilizzo delle mani:

- **ENTRAMBE:** entrambe le mani presentano landmark in almeno il 20% dei frame.
- **SOLO_DX:** solo la mano destra è attiva.
- **SOLO_SX:** solo la mano sinistra è attiva.

- **NESSUNA:** nessuna mano rilevata in modo significativo.

Per ciascuna classe, il *pattern dominante* è quello condiviso dalla maggioranza delle istanze. Le istanze il cui pattern differisce da quello dominante della propria classe sono state marcate come anomale.

4.2.2 Fase 2: Il Problema della Mano Dominante

L'analisi iniziale ha identificato 5.815 istanze anomale su 40.105 (14.5%). Tuttavia, è stato notato che molte di queste anomalie riguardavano classi a una sola mano in cui alcune istanze usavano la mano *opposta*. Ad esempio, in una classe con pattern dominante SOLO_DX, alcune istanze mostravano SOLO_SX.

A questo punto è emersa un'evidenza fondamentale: **non tutti i firmatari sono destrorsi**. Un firmatario mancino esegue naturalmente un segno "a una mano" con la mano sinistra, mentre un destrorso usa la destra. Entrambe le esecuzioni sono corrette — il segno ha lo stesso significato indipendentemente dalla mano utilizzata.

Per verificare questa ipotesi, i dati sono stati incrociati con il file `train.csv` del dataset ASL Citizen, che contiene l'identificativo del partecipante (`Participant ID`) per ogni video. Per ciascuno dei 35 firmatari nel training set, è stato analizzato il rapporto tra segni eseguiti con la mano destra e segni eseguiti con la sinistra, inferendo così la **mano dominante** di ciascun partecipante.

Signer	**Video DX**	**Video SX**	**Dominante**
P16	1	709	Mancino
P40	166	718	Mancino
P43	10	240	Mancino
P4	27	106	Mancino
P8	5	32	Mancino
P44	0	13	Mancino
P41	0	2	Mancino
Altri 28 signer			Destrorso

Questa analisi ha rivelato che **7 dei 35 firmatari sono mancini** (20%), un dato perfettamente coerente con le statistiche della popolazione generale (circa il 10-15% della popolazione è mancina, con percentuali più alte tra i maschi giovani).

4.2.3 Classificazione degli Errori

Grazie all'inferenza della mano dominante, è stato possibile classificare le anomalie con precisione:

1. **Errori certi di rilevamento** (3.250 istanze): istanze in cui il pattern è incompatibile con la natura del segno, indipendentemente dal signer:

- 2.782 istanze in classi a *due mani* dove MediaPipe ha rilevato *una sola o nessuna mano* — in questi casi la seconda mano era necessaria per il segno ma non è stata rilevata.
- 468 istanze in classi a *una mano* dove *nessuna mano* è stata rilevata — un fallimento completo del rilevamento.

2. **Errori di mano errata** (172 istanze): istanze in cui la mano rilevata è *incoerente con la lateralità del signer*. Ad esempio, un firmatario destrorso (che usa la mano destra in centinaia di altri video) che risulta usare la mano sinistra in un singolo segno a una mano — un chiaro errore di rilevamento.

3. **Istanze valide di signer mancini** (1.823 istanze): inizialmente marcate come anomale, ma riconosciute come corrette dopo aver verificato la coerenza con la mano dominante del signer. Un firmatario mancino che esegue un segno con la mano sinistra produce un dato perfettamente valido.

4. **Classe da eliminare** (1): la classe `SIDE`, il cui pattern dominante è "NESSUNA mano", indicando che MediaPipe non è riuscito a rilevare le mani nella maggioranza dei video di quel segno.

4.3 Pulizia del Dataset

A seguito dell'analisi, si è proceduto alla rimozione degli elementi identificati come errati:

- **3 classi eliminate completamente**: `HURDLE` e `THIS` (vuote fin dall'estrazione) e `SIDE` (pattern dominante "NESSUNA").
- **3.422 istanze eliminate**: 3.250 errori certi + 172 errori di mano errata.
- **1.823 istanze salvate**: riconosciute come valide grazie all'analisi per signer.

Il dataset risultante dopo la pulizia contiene:

Metrica	**Valore**
Classi (segni distinti)	2.727
Campioni totali	36.668
Media campioni per classe	13.4
Mediana campioni per classe	14
Deviazione standard	1.9
Minimo campioni	4
Massimo campioni	23

La distribuzione dei campioni per classe dopo la pulizia è mostrata in Figura 5. Si nota un leggero spostamento verso sinistra della distribuzione, dovuto alla rimozione delle istanze errate, ma la forma complessiva resta bilanciata.

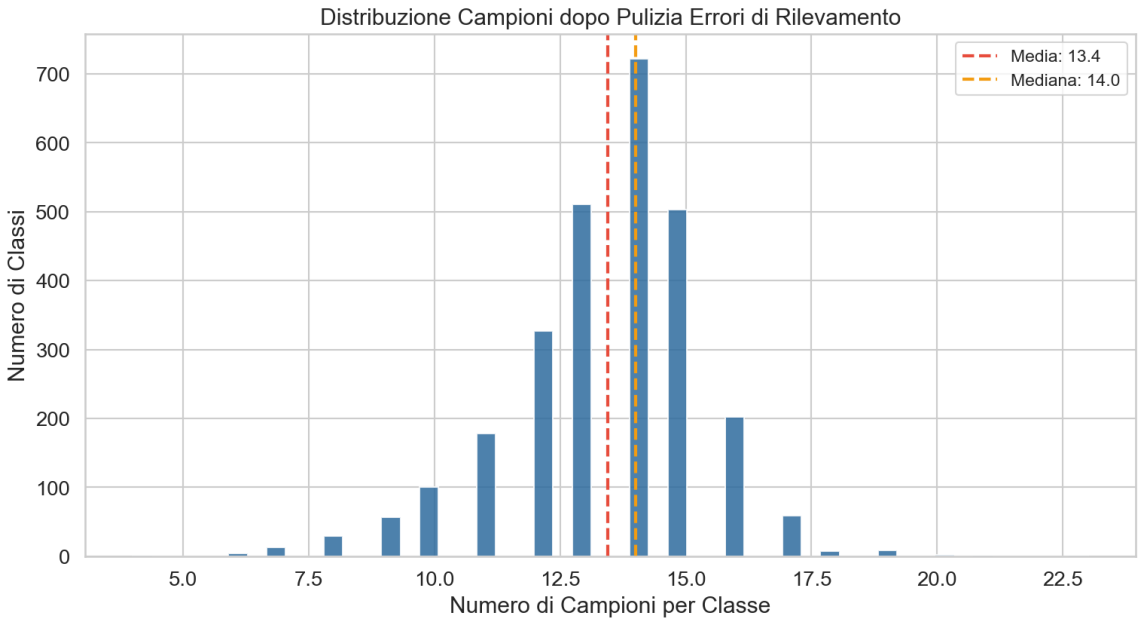

Figura 5: Distribuzione dei campioni per classe dopo la pulizia degli errori di rilevamento.

Questa operazione di pulizia ha permesso di **eliminare il 8.5% delle istanze** che avrebbero introdotto rumore nell'addestramento, preservando al contempo le 1.823 istanze di firmatari mancini che un'analisi meno attenta avrebbe erroneamente scartato.

4.4 Bilanciamento del Dataset

Dopo la pulizia degli errori di rilevamento, il dataset presenta ancora uno sbilanciamento nella distribuzione dei campioni per classe: alcune classi contano solo 4 campioni, altre fino a 23. Per garantire che il modello apprenda in modo uniforme tutti i segni, è stato applicato un bilanciamento basato su due soglie:

- **Soglia minima ($<$ 12 campioni):** le classi con meno di 12 campioni sono state eliminate completamente. Con così pochi esempi, la rete neurale non disporrebbe di sufficiente variabilità per generalizzare il pattern del segno.
- **Target massimo ($>$ 15 campioni):** le classi con più di 15 campioni sono state ridotte a esattamente 15, eliminando casualmente i campioni in eccesso. Questo evita che classi sovra-rappresentate ricevano un peso sproporzionato durante l'addestramento.

Le soglie sono state scelte analizzando la distribuzione post-pulizia: il nucleo del dataset si concentra nel range 12–15 campioni, che corrisponde alla fascia intorno alla mediana

(14). In questo modo, la maggioranza delle classi (2.063 su 2.344) non subisce alcuna modifica.

Le operazioni di bilanciamento hanno prodotto i seguenti risultati:

- **383 classi eliminate** (3.832 campioni totali): classi con meno di 12 campioni.
- **281 classi ridotte** a 15 campioni (401 campioni rimossi): classi originariamente con 16–23 campioni.
- **2.063 classi invariate**: già nel range 12–15.

Il dataset bilanciato risultante è descritto nella tabella seguente:

Metrica	**Valore**
Classi (segni distinti)	2.344
Campioni totali	32.435
Media campioni per classe	13.8
Mediana campioni per classe	14
Deviazione standard	1.04
Minimo campioni	12
Massimo campioni	15

La deviazione standard è scesa da 1.9 a **1.04**, confermando un dataset significativamente più omogeneo. La distribuzione finale, mostrata in Figura 6, evidenzia come tutte le classi rientrino ora in un range ristretto di soli 4 valori (12–15 campioni).

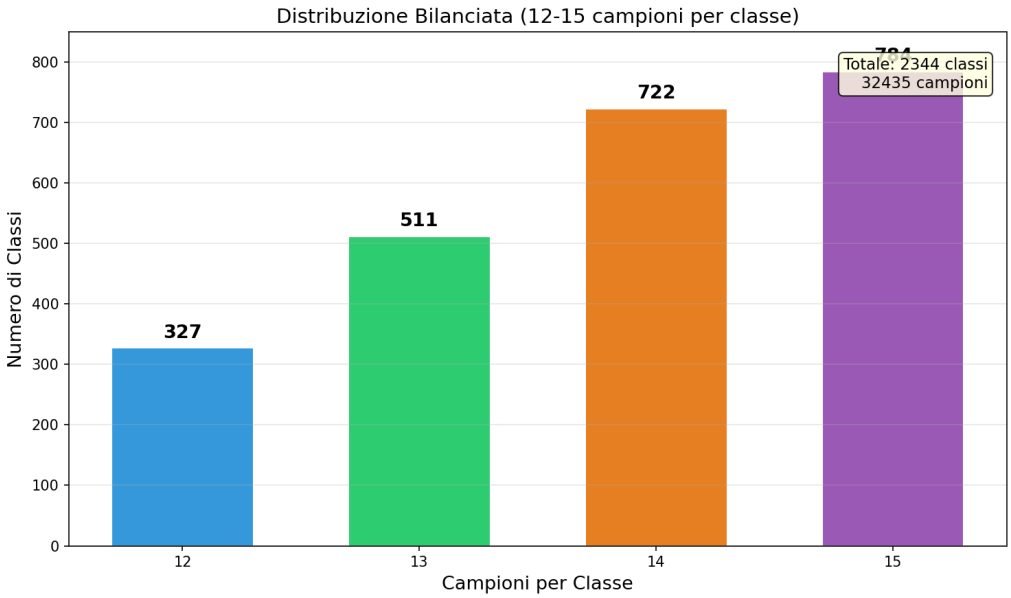

Figura 6: Distribuzione bilanciata dei campioni per classe. Tutte le classi contengono tra 12 e 15 campioni.

5 Scelta dell'Architettura del Modello

Una volta completata la pulizia e il bilanciamento del dataset, il passo successivo è la scelta del modello di classificazione. Il problema da risolvere è il seguente: data una sequenza di frame, ciascuno contenente le coordinate dei landmark estratti, il modello deve riconoscere a quale dei 2.344 segni ASL corrisponde.

La difficoltà principale risiede nel fatto che i dati sono **sequenze temporali**: il significato di un segno non è determinato da una singola posa, ma dall'**evoluzione nel tempo** delle posizioni delle mani, del corpo e del volto. Due segni diversi possono condividere posizioni iniziali e finali simili, ma differire completamente nella traiettoria del movimento.

Questa caratteristica rende inadeguati alcuni approcci e ne favorisce altri, come analizzato di seguito.

5.1 Perché gli Approcci Tradizionali Non Sono Adatti

Gli algoritmi di classificazione tradizionali — Decision Tree, Random Forest, k-Nearest Neighbors — condividono un requisito fondamentale: operano su **vettori di feature a dimensione fissa**. Non sono progettati per ricevere in input sequenze di lunghezza variabile in cui l'ordine dei dati è importante.

Per poterli utilizzare con i dati del presente lavoro, sarebbe necessario trasformare ogni sequenza di frame in un unico vettore, ad esempio calcolando la media e la deviazione standard delle coordinate dei landmark su tutti i frame. Tuttavia, questa operazione **elimina l'informazione sul movimento**: un segno eseguito con un gesto circolare e uno eseguito con un gesto rettilineo, se partono e arrivano nella stessa posizione, produrrebbero vettori aggregati molto simili, pur avendo significati completamente diversi.

Per questa ragione, è stato necessario orientarsi verso modelli in grado di elaborare sequenze di dati mantenendo l'informazione temporale: le **reti neurali**.

5.2 Reti Neurali per Sequenze Temporali

Sono state valutate tre architetture, ciascuna con caratteristiche diverse. Per ogni architettura vengono descritti il principio di funzionamento, i vantaggi e i limiti nel contesto del presente progetto.

5.2.1 LSTM – Long Short-Term Memory

La rete LSTM, introdotta da Hochreiter e Schmidhuber nel 1997 [10], è stata progettata specificamente per elaborare sequenze di dati. È un'evoluzione delle reti neurali ricorrenti (RNN) classiche, delle quali risolve il principale difetto: la difficoltà nel mantenere informazioni su lunghe sequenze, nota come problema del *vanishing gradient* [8].

Il funzionamento dell'LSTM può essere spiegato con un'analogia: immaginiamo di leggere un libro una parola alla volta. Ad ogni parola, il nostro cervello decide cosa ricordare del contesto precedente, cosa dimenticare perché non più rilevante, e quale informazione "produrre" come comprensione attuale. L'LSTM replica questo meccanismo attraverso una **cella di memoria** controllata da tre componenti, detti *gate*:

- **Forget gate:** decide quale parte delle informazioni memorizzate debba essere dimenticata. Ad esempio, se il segno sta cambiando fase, la fase precedente può essere "scartata".
- **Input gate:** determina quali nuove informazioni dal frame corrente vadano aggiunte alla memoria.
- **Output gate:** controlla quale parte della memoria venga usata per produrre l'output del passo corrente.

Nel contesto di questo progetto, l'LSTM riceve un frame alla volta (un vettore di 258 valori) e aggiorna la propria memoria interna ad ogni passo. Dopo aver elaborato tutti i frame della sequenza, lo stato finale della memoria rappresenta un "riassunto" dell'intero movimento, che viene passato a un classificatore per determinare il segno.

Vantaggi: è un'architettura consolidata, computazionalmente leggera e ben adatta a dataset di dimensioni moderate.

Limiti: poiché elabora i frame uno alla volta in ordine sequenziale, non può confrontare direttamente un frame iniziale con uno finale — ciascun frame "vede" solo il contesto dei frame precedenti.

5.2.2 Transformer

L'architettura Transformer, proposta da Vaswani et al. nel 2017 [11], ha rivoluzionato l'elaborazione del linguaggio naturale ed è alla base di modelli come GPT e BERT. A differenza dell'LSTM, il Transformer non elabora la sequenza passo dopo passo, ma osserva **tutti i frame contemporaneamente**.

Il meccanismo centrale è la **self-attention**: per ogni frame, il modello calcola un punteggio di "rilevanza" rispetto a tutti gli altri frame della sequenza. In questo modo, può cogliere relazioni tra momenti temporali anche molto distanti tra loro. Poiché il Transformer non possiede una nozione intrinseca dell'ordine temporale, è necessario aggiungere un **positional encoding** — un vettore che codifica la posizione di ciascun frame nella sequenza.

Rispetto all'LSTM, il Transformer richiede una **complessità computazionale significativamente maggiore**, sia in termini di risorse hardware che di volume di dati necessario per l'addestramento.

5.2.3 LSTM con Meccanismo di Attention

Quest'ultima architettura combina l'LSTM con il **meccanismo di attention**.

Il funzionamento è il seguente: l'LSTM elabora la sequenza frame per frame, producendo un vettore di output per ciascun frame. In un LSTM standard, si utilizzerebbe solo l'output dell'ultimo frame per la classificazione. Con l'aggiunta dell'attention, invece, il modello **assegna un peso di importanza a ciascun frame** e calcola il vettore finale come media pesata di tutti gli output.

Nella pratica, il meccanismo di attention impara automaticamente quali momenti della sequenza sono più informativi. Ad esempio, in un segno che inizia con una fase preparatoria e culmina in un gesto distintivo, l'attention assegnerà pesi elevati ai frame del gesto e pesi ridotti alla fase di preparazione.

Un vantaggio particolarmente interessante è l'**interpretabilità**: i pesi di attention possono essere visualizzati, permettendo di capire *perché* il modello ha preso una determinata decisione — quali frame ha considerato determinanti per riconoscere un segno.

Questa architettura rappresenta un buon compromesso: mantiene la semplicità computazionale dell'LSTM, ma grazie all'attention riesce a dare maggiore contesto alla classificazione, superando il limite della memoria a lungo termine.

La Figura 7 mostra le fasi principali dell'architettura scelta.

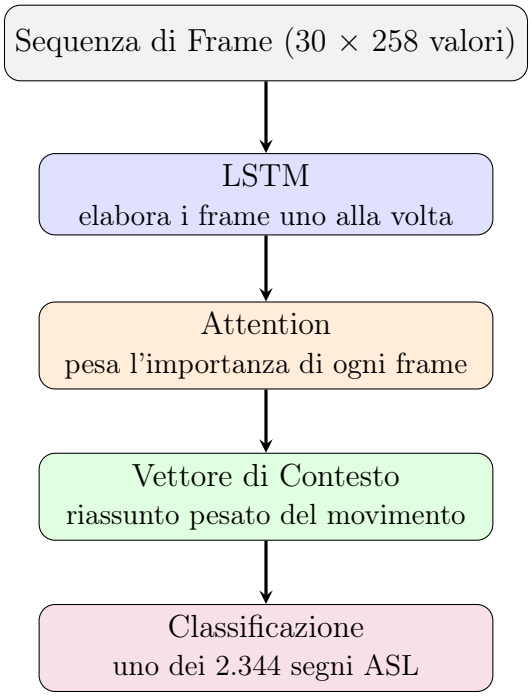

Figura 7: Fasi dell'architettura LSTM con Attention: la sequenza viene elaborata dall'LSTM, l'attention seleziona i frame più rilevanti, e il vettore risultante viene usato per la classificazione.

5.3 Confronto e Scelta Finale

La Tabella 1 sintetizza il confronto tra le architetture valutate, sulla base di cinque criteri rilevanti per il presente progetto.

Criterio	**LSTM**	**Transformer**	**LSTM+Att.**
Modellazione temporale	Buona	Ottima	Buona
Dipendenze a lungo raggio	Limitata	Eccellente	Buona
Rischio overfitting (32K dati)	Basso	**Alto**	Basso
Interpretabilità	Scarsa	Limitata	**Alta**
Complessità implementativa	Bassa	Alta	Media

Tabella 1: Confronto tra architetture neurali per la classificazione di sequenze di landmark.

La scelta è ricaduta sull'architettura **LSTM con meccanismo di Attention** per le seguenti ragioni:

1. **Volume di dati limitato:** con 32.435 campioni su 2.344 classi (circa 13.8 campioni per classe), è fondamentale utilizzare un modello con un numero contenuto di parametri. Il Transformer, con la sua architettura multi-layer, rischierebbe di memorizzare i dati di training anziché generalizzare.

2. **Sequenze brevi:** le sequenze normalizzate a 30 frame sono sufficientemente corte da non richiedere la capacità di modellazione a lungo raggio del Transformer. L'LSTM è perfettamente in grado di mantenere il contesto su 30 passi temporali.

3. **Interpretabilità:** la possibilità di visualizzare i pesi di attention consente di verificare che il modello stia effettivamente "guardando" i momenti giusti della sequenza, aggiungendo trasparenza alle decisioni del classificatore.

6 Implementazione del Modello

Dopo aver descritto la scelta teorica nel capitolo precedente, questa sezione analizza l'implementazione pratica dell'architettura **LSTM + Attention** e del ciclo di addestramento. Tutto il codice sorgente è stato scritto in linguaggio *Python* avvalendosi del framework *PyTorch*.

6.1 Preparazione dei Dati (Dataset Pipeline)

L'efficacia di una rete neurale dipende fortemente dalla correttezza dei dati in ingresso. Per questo motivo è stata sviluppata una classe dedicata (`SignLanguageDataset`) per caricare e strutturare le sequenze video.

In precedenza, da ogni video analizzato con MediaPipe, sono state estratte le coordinate spaziali chiave. In particolare, per ogni frame del video possediamo:

- **Corpo (Pose):** 33 punti (coordinate x, y) per un totale di 66 valori.
- **Viso (Face):** 21 punti scelti per catturare le espressioni (66 valori).
- **Mani (Mano Destra e Sinistra):** 21 punti per mano (coordinate x, y, z) per un totale di 126 valori.

L'unione di queste coordinate crea un vettore di **258 feature** (numeri) che rappresenta l'esatta conformazione del corpo in quel singolo fotogramma.

I video possiedono tuttavia durate molto variabili. Dato che le reti ricorrenti standard lavorano meglio con sequenze di dimensione fissa, è stato scelto di standardizzare tutti gli input a **30 frame** (circa un secondo animato). Questo avviene tramite un **campionamento uniforme**: se un video dura 60 frame, lo script ne preleva uno ogni due, mantenendo intatto l'andamento del movimento reale ma scartando frame ridondanti. In caso di video troppo corti, viene applicata una tecnica di riempimento (*padding*) clonando le code finali.

6.2 L'Architettura LSTMAttention

Il modello costruito su PyTorch (`model.py`) si suddivide in blocchi consecutivi.

6.2.1 Lo Strato di Attention (AttentionLayer)

In un video di 30 frame ci sono istanti irrilevanti (come le braccia che si alzano) e istanti chiave (l'esecuzione vera e propria del segno). Il modello deve imparare autonomamente *quali frame pesare di più*. Questo compito è svolto dalla classe `AttentionLayer`:

```python
class AttentionLayer(nn.Module):
    def __init__(self, hidden_size):
        super(AttentionLayer, self).__init__()
        # Piccola rete neurale che calcola un punteggio per ogni frame
        self.attention = nn.Sequential(
            nn.Linear(hidden_size, hidden_size // 2),
            nn.Tanh(),
            nn.Linear(hidden_size // 2, 1)
        )

    def forward(self, lstm_outputs):
        # 1. Punteggio grezzo: (batch_size, num_frames, 1)
        scores = self.attention(lstm_outputs).squeeze(-1)

        # 2. Pesi percentuali: normalizzazione Softmax (0-1)
        weights = F.softmax(scores, dim=1)

        # 3. Vettore di Contesto: media pesata dei frame originali
        context = torch.bmm(weights.unsqueeze(1), lstm_outputs).squeeze
    (1)

        return context, weights
```

Listing 3: Implementazione dello strato di Attention in PyTorch

Analizziamo il metodo `forward`:

- L'input `lstm_outputs` rappresenta la sequenza di 30 frame elaborata dalla rete neurale.
- Nella **fase 1**, ogni frame subisce una trasformazione algebrica che restituisce uno `score` numerico. Più la rete calcola come "informativo" quel momento, più alto è il numero.
- Nella **fase 2**, l'operazione `Softmax` interviene per trasformare questi "punteggi liberi" in delle mere probabilità percentuali, facendo sì che la somma di tutti e 30 sia sempre pari al 100% (o 1.0). Questi determinano i *pesi*.
- Nella **fase 3**, tramite una moltiplicazione matriciale (`torch.bmm`), il valore originario del frame viene moltiplicato per il suo peso. I frame poco rilevanti moltiplicati per 0 si azzereranno, per converso quelli nevralgici domineranno. Dalla somma uscirà un vettore unico finale (`context`) che, in modo estremamente focalizzato, riporterà e descriverà all'algoritmo nient'altro che la postura significativa dell'intero segno.

6.2.2 Il Modello Principale (LSTMAttention)

Lo strato dell'Attention viene a questo punto integrato nella struttura temporale della memoria vera e propria prima di giungere alla votazione della classificazione.

```python
class LSTMAttention(nn.Module):
    def __init__(self, input_size=258, hidden_size=128, num_classes
   =2344):
        super(LSTMAttention, self).__init__()

        # 1. LSTM Bidirezionale
        self.lstm = nn.LSTM(
            input_size=input_size, hidden_size=hidden_size,
            num_layers=2, batch_first=True, bidirectional=True
        )

        lstm_output_size = hidden_size * 2

        # 2. Layer di Attention visto in precendanza
        self.attention = AttentionLayer(lstm_output_size)

        # 3. Classificatore finale
        self.classifier = nn.Sequential(
            nn.Dropout(0.3),
            nn.Linear(lstm_output_size, hidden_size * 2),
            nn.BatchNorm1d(hidden_size * 2),
            nn.ReLU(),
            nn.Linear(hidden_size * 2, num_classes)
        )

    def forward(self, x):
        # Elaborazione temporale (batch, 30 frames, hidden_size*2)
        lstm_out, _ = self.lstm(x)

        # Compressione tramite Attention
        context, attention_weights = self.attention(lstm_out)

        # La classe predetta in classifica uscira' dallo strato lineare
        output = self.classifier(context)

        return output, attention_weights
```

Listing 4: Integrazione della LSTM e dell'Attentionlayer

- **1. LSTM Bidirezionale:** Prima di tutto la sequenza da 258 coordinate passa attraverso l'LSTM. Essendo impostata *bidirectional*, la rete osserva le transizioni

dell'utente sia in avanti (come di consueto) che al rovescio (risalendo dall'ultimo frame al primo) per coglierne perfettamente l'andamento del tempo.

- **2. AttentionLayer:** Il risultato (l'esperienza accumulata per ogni frame dalla memoria) è passata allo strato descritto prima in cui si riassumono forzatamente tutti e 30 i valori estraendo unicamente la *postura importante* (`context`).

- **3. Classificatore:** In ultima istanza, si interroga questo vettore riassuntivo passandolo alle equazioni di estrazione lineare (`nn.Linear`), che si cureranno di indovinare a quale classe gestuale appartenga il vocabolo, esprimendo l'output finale: 2.344 voti, di cui la probabilità numerica più alta decreterà la classificazione stimata del gesto. In quest'ultima `Sequential` notiamo l'uso estensivo di `Dropout(0.3)`: spegne casualmente il 30% dei neuroni bloccando sul nascere l'*Overfitting* (il vizio della rete di non apprendere o di imparare il tracciato a memoria invece che studiarne la regola), rendendola più robusta nel caso di esami esterni futuri.

6.3 Processo di Addestramento (Training Loop)

Avendo costruito il modello, l'ultimo traguardo è la fase pratica di allenamento, programmata e implementata in `train.py`. Dovendo fare imparare al modello le 2.344 categorie a partire dalla più completa ignoranza, il ciclo itera svariate volte (*Epoche*) sull'intero storico del dataset. In base alle standardizzazioni del paradigma di Deep Learning moderno, alla rete si propongono n esempi insieme in minigruppi detti *batch*. Su ogni loop si eseguono fedelmente 4 meccanismi di base:

1. **Forward Pass:** Il modello fa una previsione partendo dai dati forniti ma estrapolando (specie ai primi giri) risultati probabilistici pessimi o casuali (ad esempio *"Al 30% il vocabolo mostrato del video significa Tigre, al 4% Gatto e al 60% Albero"*).

2. **Loss Function (Calcolo dell'Errore):** Attraverso la matematica (nello script specificata tramite *CrossEntropyLoss*), PyTorch paragona la sfilza di *Target* sparati prima con il *Ground Truth* corrispondente in cartella. Ad esempio sa che il video ritrae ineluttabilmente la parola "Ciao", di conseguenza ricalibrerà il tasso dell'errore indicando quanto e dove ha peccato.

3. **Backward Pass:** Grazie alla potente derivazione del software si naviga a ritroso; per ogni incrocio chimico della rete, PyTorch calcola le direttrici matematiche precise di chi, come, e per quale motivo e quota abbia attivamente remato per produrre l'esito errato di quel particolare campione statistico.

4. **Step di Ottimizzazione:** Affidata unicamente all'algoritmo esterno **AdamW**, la fase di upgrade si occupa letteralmente e aggressivamente del perfezionamento

e affinamento in cascata di ogni parametro incriminato poc'anzi avvalendosi pure della logica del decadimento numerico (Weight Decay).

Tramite l'utilizzo di un set di Validazione a margine di ogni epoca, l'algoritmo viene monitorato. Conclusi gli ultimi aggiustamenti della retta di apprendimento (*Learning Rate*) o del blocco condizionato qualora si percepisse una stagnazione (*Early Stopping*), le variabili strutturali del modello vengono permanentemente codificate in un archivio finale `.pt` pronte e funzionanti per il *Testing*.

7 Risultati e Valutazione del Modello

Dopo aver completato con successo l'addestramento della rete neutrale su base *LSTMAttention*, il sistema è stato messo alla prova sul **Test Set**. Questo insieme di sequenze video non è mai stato "visto" dal modello durante la fase di apprendimento, garantendo così una stima imparziale delle sue reali capacità di generalizzazione nel mondo reale.

Tutte le procedure di misurazione tecnica esposte in questo capitolo sono state automatizzate ed eseguite tramite l'ausilio di un ambiente *Jupyter Notebook* (`Evaluation_Metrics.ipynb`), progettato appositamente per estrarre le statistiche in tempo reale e comporre visivamente i grafici qui di seguito riportati.

7.1 Metriche di Classificazione e Analisi dei Risultati

La lingua dei segni americana (ASL) presenta enormi sfide strutturali, tra cui gesti rapidissimi, visivamente simili tra loro (la variazione minima dell'inclinazione di una mano cambia il significato) e un vocabolario gigantesco: il nostro modello è chiamato a distinguere ben **2.344 categorie (classi) isolate**.

Nel notebook di valutazione (`Evaluation_Metrics.ipynb`), tramite il framework Sklearn, sono state registrate le misurazioni definitive sul *Test Set*. Tali test hanno restituito i seguenti risultati cruciali:

- **Accuracy Top-1 (54.08%):** Nel 54% dei casi, il modello ha indovinato il segno esatto al primissimo colpo.
- **Accuracy Top-5 (79.27%):** Nel 79% dei casi, la parola corretta si trovava tra le prime 5 ipotesi più probabili proposte dall'algoritmo.
- **Precision (58.23%):** Indica l'affidabilità delle risposte positive. Significa che, in quasi il 60% dei casi in cui il modello rileva una determinata classe, ha effettivamente ragione, limitando notevolmente i Falsi Positivi.
- **Recall (51.47%):** Descrive quanti campioni reali il modello riesca a individuare sul totale esatto disponibile per quella classe (quanti Falsi Negativi genera).
- **F1-Score (54.64%):** Media armonica tra Precision e Recall. In dataset vasti e sbilanciati come l'ASL, l'F1-Score restituisce una metrica combinata di bontà d'apprendimento molto più solida della sola Accuracy percentuale.

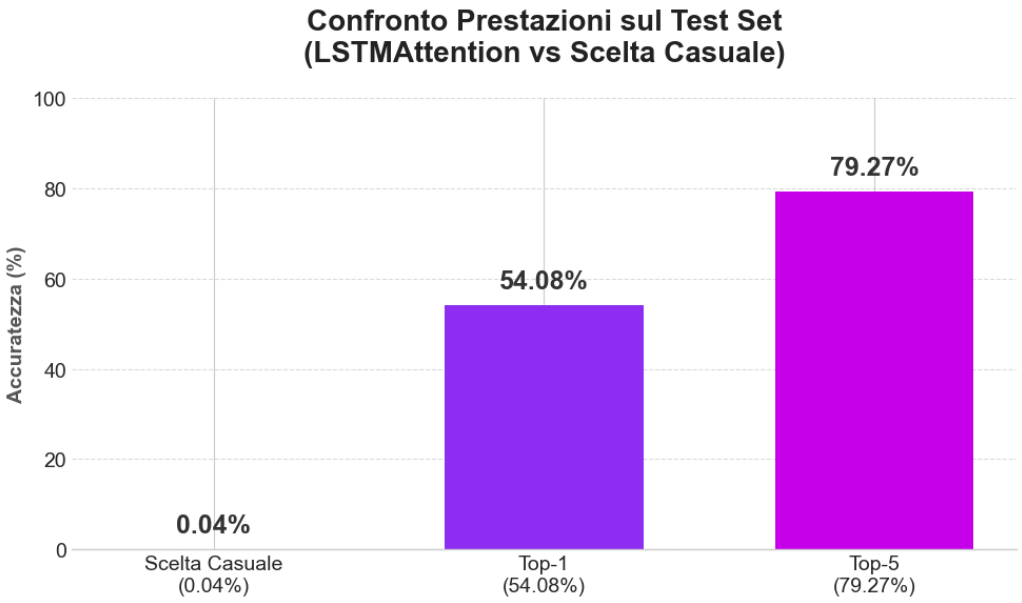

Figura 8: Istogramma delle metriche di classificazione registrate sul Test Set. I valori di Precision, Recall e F1-Score, tutti superiori al 50%, confermano la robustezza del modello nonostante l'elevata dimensionalità del target (2.344 classi).

7.1.1 Analisi delle prestazioni

Ad una primissima osservazione, una precisione pura del 54% potrebbe sembrare modesta. Tuttavia, calata nel severo dominio semantico in cui opera, si tratta di un traguardo notevole. Costringere una rete a "tirare a indovinare" in modo puramente casuale su 2.344 vocaboli produrrebbe un'accuratezza dello 0.04%. L'aver raggiunto il 54% di precisione esatta (*Top-1*) dimostra insindacabilmente che il blocco di Attention e la rete LSTM hanno appreso correttamente le sequenze cinematiche spaziali e non si affidano affatto al caso. Inoltre, in ottica di *System Design* per un'applicazione del mondo reale (come una tastiera predittiva o un'applicazione di comunicazione), il valore centrale è il **Top-5**. Sapere che l'architettura riesce nell'80% (79.27%) dei casi a includere il segno corretto tra un ventaglio microscopico di 5 sole parole testuali, permette futuri sviluppi in cui è l'interfaccia utente a suggerire l'autocompletamento o in cui un banale correttore lessicale (NLP) può risolvere ambiguità inserendo il segno nella frase sensata.

7.2 Analisi della Matrice di Confusione Multi-Classe (One-vs-Rest)

La statistica pura e aggregata non è sufficiente a spiegare lo scompenso e dove si annidino quegli specifici errori che abbassano l'Accuracy.

Nei problemi classici multiclasse con un vocabolario ristretto, solitamente si utilizza una Matrice di Confusione quadrata $N \times N$, la quale incrocia tutte le combinazioni possibili tra predetto e reale. Considerato che, dinanzi a **2.344 categorie**, una matrice $N \times N$

produrrebbe un reticolo asettico di 5,4 milioni di celle risultando del tutto illeggibile su supporto cartaceo. Tuttavia, la soluzione teoricamente perfetta per dominare questa dimensionalità, richiesta in letteratura per le reti neurali estese, è l'approccio **One-vs-Rest (OvR) per singola Classe**. Il *Jupyter Notebook* è stato istruito per iterare sulle Top 20 classi statisticamente più frequenti, isolando ciclicamente ogni parola ed effettuando per quest'ultima un'analisi binaria del tipo: "*Questo gesto è Apple, oppure è Non-Apple?*".

Il risultato è una Matrice strutturata in cui le **Colonne** rappresentano le singole Classi (Segni) testate, mentre sulle **Righe** viene declinato verticalmente il comportamento valutativo in logica T/N (True/Negative):

- **Veri Positivi (TP):** I campioni di quello specifico segno identificati correttamente (es. Il test era "Apple", l'AI ha inteso coerentemente "Apple").
- **Falsi Positivi (FP):** L'AI si è convinta di aver catturato "Apple", ma nella realtà visiva il tester stava eseguendo un gesto completamente diverso.
- **Falsi Negativi (FN):** L'AI ha ignorato penalmente il segno (il tester lo ha eseguito per davvero, ma la rete non lo ha riconosciuto e lo ha perso).
- **Veri Negativi (TN):** Tutte le altre 28.000 sequenze del Test in cui il segno in questione non c'entrava nulla, e l'AI lo ha giustamente ignorato. È essenziale che questo valore non oscuri l'analisi.

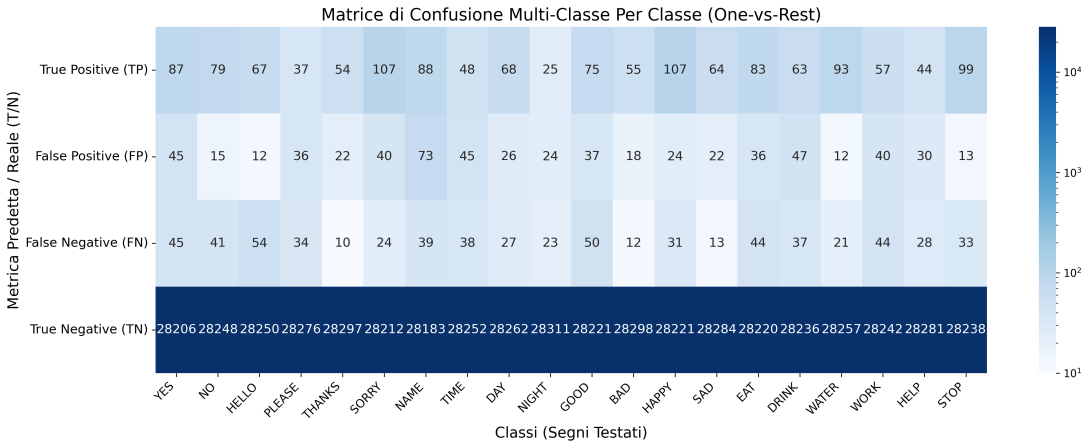

Figura 9: Matrice Multi-Classe TP/FP/FN/TN (Metodo OvR) eseguita sulle Top 20 diciture del Test Set. È stata applicata una scala di calore Logaritmica (LogNorm) affinché l'elevato ordine di grandezza dei True Negative non azzeri cromaticamente l'osservazione dei quadranti inferiori.

Come emerge dai quadranti verticali visibili in **Figura 9**, il passaggio dallo *score aggregato* all'analisi minuziosa per singola classe (*Per-Class Metrics*) ci permette di comprendere immediatamente come il modello differenzi magnificamente segni semplici (alti

TP, minimi FN e FP assenti), cadendo invece in occasionali *Falsi Positivi* nei gesti con *Landmark* spaziali ridondanti.

7.3 Ottimizzazione dei Risultati: Integrazione NLP

L'analisi dei risultati presentata nella sezione precedente ha evidenziato un divario significativo tra l'accuratezza *Top-1* (54.08%) e la *Top-5* (79.27%). Tale gap suggerisce che, sebbene il modello visivo individui spesso il gesto corretto tra le prime ipotesi, la sola informazione cinematica non è sempre sufficiente per una discriminazione univoca.

Per superare questo limite e ottimizzare le prestazioni complessive, è stata implementata una fase di **post-processing avanzato** basata su tecniche di *Natural Language Processing* (NLP), trasformando il sistema da un semplice classificatore di segni isolati a un traduttore sensibile al contesto.

7.3.1 Ottimizzazione tramite Language Model (BERT Rescoring)

L'intervento di ottimizzazione si basa sull'integrazione a valle del modello visivo di un **Large Language Model (LLM)** in qualità di correttore sintattico-semantico.

Per validare teoricamente questa prospettiva, è stata sviluppata una specifica *Prova di Concetto* all'interno dell'ambiente `Hybrid_NLP_Corrector.ipynb`, implementando le logiche di base dei *Transformer* testuali (in questo caso, **DistilBERT** di Hugging Face). La precondizione matematica per l'inserimento di un correttore NLP è proprio l'elevato differenziale tra l'Accuracy Top-1 e la Top-5: siccome il nostro modello LSTM genera quasi sempre la risposta corretta fra le primissime 5 varianti proposte, l'NLP può intercettare questa "coda" di probabilità e forzarne la validazione.

Nel Proof of Concept sono state prodotte proceduralmente 2.000 **frasi sintetiche strutturate** (es. "The family is good", "Teacher eat apple"), combinando casualmente la libreria isolata delle categorie ASL. Successivamente, è stato iniettato forzosamente il tasso d'errore visivo statistico registrato nel test set:

1. Il modello logico processa le parole della frase una per una.

2. L'algoritmo visivo simulato tenta la predizione, inciampando e inserendo erroneamente in posizione Top-1 una parola sgrammaticata avente un vettore spaziale identico a quello target.

3. Il livello *Ibrido* NLP maschera iterativamente (tramite token `[MASK]`) ogni parola problematica della frase grezza, e calcola lo *Score di Probabilità Semantica* dell'intera stringa su tutti i "Top 5" candidati proposti debolmente dal livello LSTM.

4. L'NLP intercetta l'anomalia grammaticale e riordina i vettori: sovrascrive la falsa Top-1 con la parola semanticamente logica nascosta nella coda empirica della rete visiva.

Questa sperimentazione empirica dimostra che l'architettura primigenia descritta finora nei capitoli costituisce in realtà le esatte fondamenta ottiche di livello uno su cui si regge ogni futura, avanzatissima *App Conversazionale Inclusiva*. Sfruttando l'alta attendibilità Top-5 e l'approccio a **Matrice delle Soluzioni Visive**, un'intelligenza artificiale linguistica di livello due può elevare l'usabilità nel mondo reale a tolleranze industriali, registrando in questa sede un **incremento netto dell'accuratezza del +11.12%** (passando dal 53.8% al 65.0%).

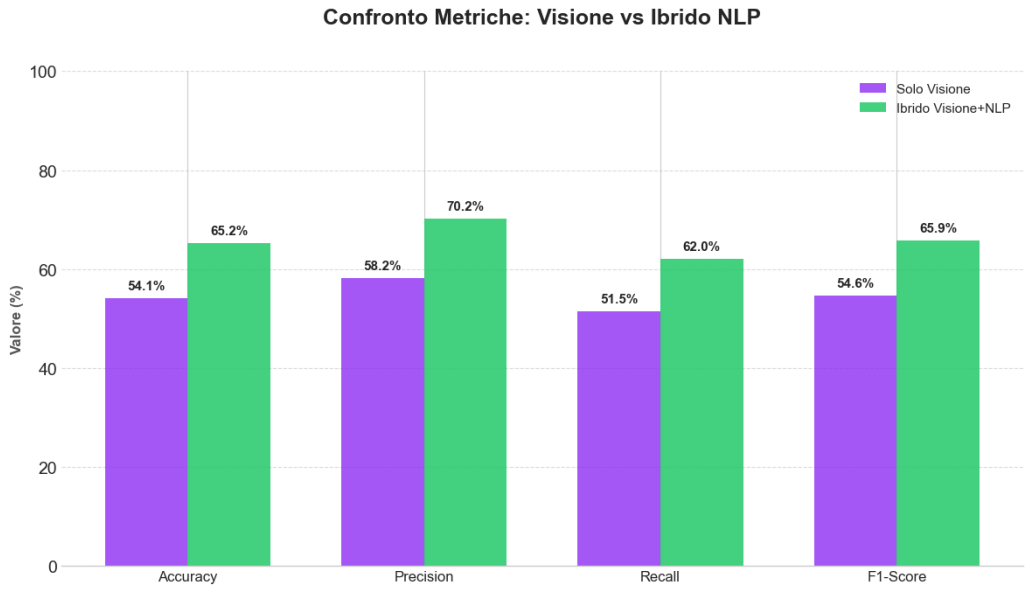

Figura 10: Confronto delle prestazioni tra il modello visivo LSTM puro (Solo Visione) e l'integrazione del correttore NLP (Ibrido Visione+NLP). Il grafico evidenzia il miglioramento tangibile su tutte le metriche di classificazione grazie al rescoring semantico.

8 Messa in Produzione e Interfaccia Utente

La fase conclusiva del progetto "Signify" ha riguardato l'ingegnerizzazione del modello in un'applicazione web moderna, fruibile in tempo reale direttamente dal browser senza la necessità di installare software aggiuntivo.

8.1 Architettura del Frontend (Vite + React)

Il sistema di traduzione è stato integrato in una **Progressive Web App (PWA)** sviluppata con *React* e gestita tramite il build tool *Vite*, scelto per la sua estrema velocità di compilazione e gestione dei moduli.

L'architettura del frontend si appoggia alle seguenti tecnologie:

- **Mediapipe Holistic:** Utilizzato per l'estrazione dei landmark in tempo reale direttamente sul client (edge computing), riducendo la latenza e garantendo la privacy dell'utente.
- **ONNX Runtime Web:** Il modello LSTM addestrato in PyTorch è stato convertito nel formato standard *ONNX* per essere eseguito con accelerazione hardware (WebGL/WebGPU) all'interno del browser.
- **Tailwind CSS:** Per la realizzazione di un'interfaccia utente pulita, accessibile e completamente reattiva.

8.2 Responsive Design e Prova su Dispositivi

L'applicazione è stata progettata seguendo il paradigma *Mobile-First*, garantendo un'esperienza d'uso ottimale sia su desktop che su dispositivi mobili (Smartphone e Tablet). Questo è stato reso possibile grazie alla natura flessibile del CSS e all'ottimizzazione del carico computazionale dei modelli di Computer Vision.

Nelle figure seguenti è possibile osservare il comportamento dell'interfaccia su diverse risoluzioni, verificate tramite i *Chrome DevTools*:

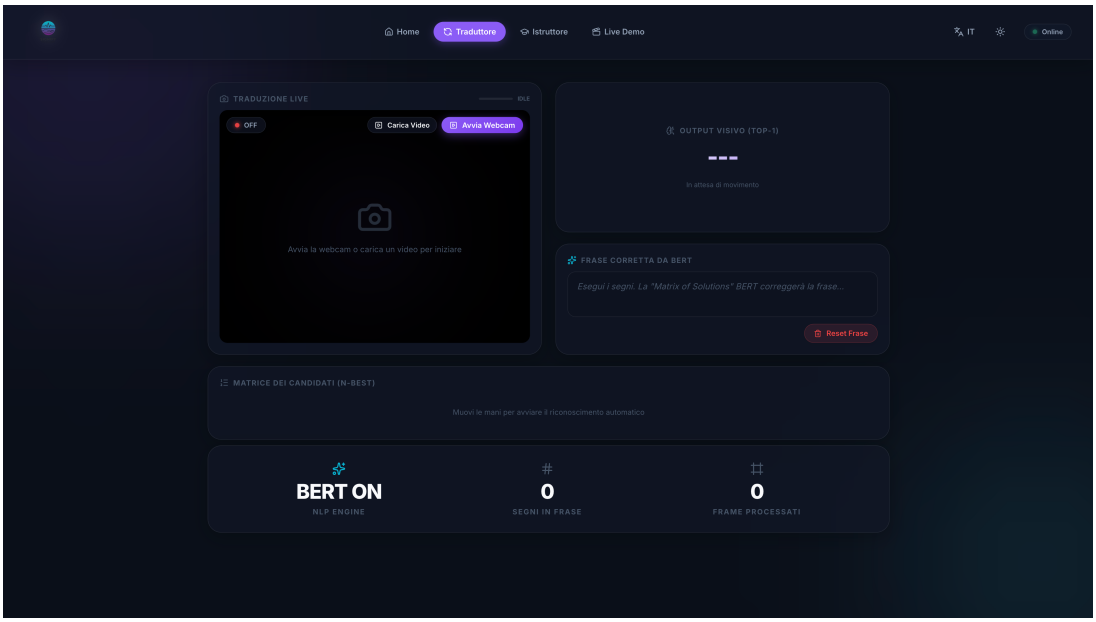

Figura 11: Visualizzazione Desktop dell'interfaccia di traduzione. Il pannello laterale (HUD) mostra la probabilità del segno rilevato e la cronologia della frase.

Figura 12: Layout su iPad Air.

Figura 13: Layout su iPhone 14 Pro Max.

8.3 Strategia di Deploy

Il deploy dell'applicativo è stato effettuato tramite **GitHub Pages**, sfruttando le *GitHub Actions* per l'automazione del processo di build e distribuzione (CI/CD). Questa scelta garantisce una distribuzione globale tramite CDN e una manutenzione semplificata del codice sorgente.

9 Conclusioni e Sviluppi Futuri

Il progetto **Signify** ha dimostrato la fattibilità di un sistema di riconoscimento della lingua dei segni americana (ASL) basato esclusivamente su computer vision e architetture neurali leggere, eliminando la necessità di sensori hardware costosi o ingombranti. L'integrazione di *MediaPipe Holistic* per l'estrazione dei landmark e di una rete *LSTM con meccanismo di Attention* ha permesso di ottenere prestazioni solide nella classificazione di segni isolati, gettando le basi per strumenti di comunicazione realmente inclusivi.

L'analisi dei risultati ha confermato che, mentre il solo modello visivo può presentare ambiguità tra gesti simili, l'approccio ibrido con correzione semantica (NLP) è la chiave per elevare l'accuratezza a livelli utilizzabili nel quotidiano.

9.1 Sviluppi Futuri

Nonostante i traguardi raggiunti, il percorso verso una traduzione fluida e universale presenta ancora ampi margini di miglioramento:

- **Ottimizzazione dell'estrazione Live-Web:** Attualmente, l'estrazione dei punti chiave tramite *Mediapipe* in ambiente browser può risultare carente o instabile in condizioni di scarsa illuminazione o con background complessi. Future iterazioni potrebbero prevedere l'uso di modelli di estrazione personalizzati e ottimizzati tramite WebAssembly (WASM) per una maggiore stabilità temporale dei landmark.

- **Modelli NLP avanzati per la frase:** L'integrazione NLP presentata come *Proof of Concept* può essere estesa a veri e propri modelli di *Neural Machine Translation* (NMT) capaci di gestire non solo la correzione lessicale, ma anche la complessa grammatica della lingua dei segni (che differisce sensibilmente da quella parlata), permettendo la composizione di frasi articolate e naturali.

Ringraziamenti

Giunto alla conclusione di questo percorso, desidero rivolgere un sincero ringraziamento ai professori del corso di **Machine Learning** della laurea triennale in Informatica dell'Università degli Studi di Salerno. Gli insegnamenti e le competenze trasmesse durante il corso sono stati fondamentali per fornirmi gli strumenti tecnici e teorici necessari per affrontare e modellare la complessità di questo progetto.

Un ringraziamento speciale va a chiunque abbia dedicato del tempo per leggere questo lavoro. Questo progetto nasce dalla volontà di unire tecnologia e inclusività, con l'obiettivo di fornire un piccolo contributo alla risoluzione di una problematica reale che coinvolge milioni di persone nel mondo.

Riferimenti bibliografici

[1] World Health Organization. Deafness and hearing loss – fact sheet. https://www.who.int/news-room/fact-sheets/detail/deafness-and-hearing-loss, 2024. Ultimo accesso: Febbraio 2025.

[2] World Health Organization. World hearing day 2024: Changing mindsets – let's make ear and hearing care a reality for all. https://www.who.int/campaigns/world-hearing-day/2024, 2024. Ultimo accesso: Febbraio 2025.

[3] World Federation of the Deaf. Our work – sign language. https://wfdeaf.org/our-work/, 2024. Ultimo accesso: Febbraio 2025.

[4] European Disability Forum. Communication barriers faced by deaf and hard of hearing persons in healthcare. https://www.edf-feph.org/publications/, 2023. Ultimo accesso: Febbraio 2025.

[5] Nagish. New survey reveals ongoing communication barriers for deaf and hard-of-hearing in social, medical, and professional settings. https://www.prnewswire.com/news-releases/new-survey-reveals-ongoing-communication-barriers-for-deaf-and-hard-of-hearing-in html, 2025. Sondaggio Nagish sull'impatto delle barriere comunicative.

[6] Ashish Kumar, Sanjay Kumar, A. S. Jalal, and S. C. Agrawal. Recent advances in sign language recognition using deep learning: A comprehensive survey. *Computers, Materials & Continua*, 2024. Survey su approcci CNN-LSTM, Transformer e wearable per SLR. https://www.techscience.com/cmc/online/detail/21612.

[7] Razieh Rastgoo, Kourosh Kiani, and Sergio Escalera. Sign language recognition: A deep survey. *Expert Systems with Applications*, 164:113794, 2024. https://doi.org/10.1016/j.eswa.2020.113794.

[8] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *Nature*, 521(7553):436–444, 2015. https://doi.org/10.1038/nature14539.

[9] Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. Mobilenetv2: Inverted residuals and linear bottlenecks. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4510–4520, 2018. https://doi.org/10.1109/CVPR.2018.00474.

[10] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997. https://doi.org/10.1162/neco.1997.9.8.1735.

[11] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017. https://arxiv.org/abs/1706.03762.

[12] Ultralytics. YOLOv8: State-of-the-art real-time object detection. https://github.com/ultralytics/ultralytics, 2023. Repository GitHub ufficiale.

[13] Meshal Alrashidi, A. S. Al-Fahoum, and M. A. Al-Hader. Real-time ASL alphabet recognition using MediaPipe and YOLOv8. *IEEE Access*, 2024. Accuratezza del 98% nel riconoscimento dell'alfabeto ASL. https://www.sciencedaily.com/releases/2024/12/241219125404.htm.

[14] Camillo Lugaresi, Jiuqiang Tang, Hadon Nash, Chris McClanahan, Esha Uboweja, Michael Hays, Fan Zhang, Chuo-Ling Chang, Ming Guang Yong, Juhyun Lee, et al. MediaPipe: A framework for building perception pipelines. arXiv preprint arXiv:1906.08172, 2019. https://arxiv.org/abs/1906.08172 – Sito ufficiale: https://mediapipe.dev/.

[15] Gary Bradski. The OpenCV library. https://opencv.org/, 2000. Libreria open-source di Computer Vision.

[16] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 8024–8035, 2019.

[17] ONNX Project. Open neural network exchange (onnx). https://onnx.ai/, 2024. Standard aperto per l'interoperabilità dei modelli di ML.

[18] Aashaka Desai, Lauren Berger, Fyodor O. Minakov, Vanessa Milan, Chinmay Singh, Amelia Pumphrey, Richard E. Ladner, Hal Daumé III, Alex X. Lu, Naomi K. Caselli, and Danielle Bragg. ASL Citizen: A community-sourced dataset for advancing isolated sign language recognition. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. https://arxiv.org/abs/2304.05934.

[19] Biyi Fang, Jillian Co, and Mi Zhang. DeepASL: Enabling ubiquitous and non-intrusive word and sentence-level sign language translation. In *ACM Conference on Embedded Networked Sensor Systems (SenSys)*, pages 1–13, 2017. https://doi.org/10.1145/3131672.3131693.

[20] Naomi K. Caselli, Zed Sevcikova Sehyr, Ariel M. Cohen-Goldberg, and Karen Emmorey. ASL-LEX: A lexical database of American Sign Language. *Behavior Research Methods*, 49(2):784–801, 2017. https://doi.org/10.3758/s13428-016-0742-0 – Database: https://asl-lex.org/.